

Zoho Software Developer Interview – August 2024

Detailed Step-by-Step Solutions with Explanations

Section 1 – Aptitude

Q1. A tortoise at the bottom of a 65 m hill climbs 5 m every hour and slips down 3 m. How long to reach the top?

What is given:

- 65 m hill
- Climbs 5 m every hour
- Slips back 3 m every hour

Key Idea: *In the LAST hour the tortoise reaches the top and does NOT slip back. So treat the last hour separately.*

Step-by-Step:

Step 1: Net gain per hour = $5 - 3 = 2$ m

Step 2: But in the final hour it climbs 5 m and stops. So keep 5 m aside: $65 - 5 = 60$ m

Step 3: Time to cover 60 m at 2 m/hour = $60 / 2 = 30$ hours

Step 4: 1 more hour for the final 5 m climb

Step 5: Total = $30 + 1 = 31$ hours

Answer: 31 hours

Q2. $f(x + y) = f(x) + f(y)$ for all positive integers. If $f(2) = 4$, find $f(3)$.

What is given:

- Rule: $f(x + y) = f(x) + f(y)$
- $f(2) = 4$, find $f(3)$

Key Idea: *The rule says: split any number into two parts and add their f values.*

Step-by-Step:

Step 1: Write $f(2)$ using the rule: $f(2) = f(1 + 1) = f(1) + f(1)$

Step 2: We know $f(2) = 4$, so: $f(1) + f(1) = 4 \rightarrow 2 \times f(1) = 4 \rightarrow f(1) = 2$

Step 3: Now write $f(3)$ using the rule: $f(3) = f(2 + 1) = f(2) + f(1)$

Step 4: $f(3) = 4 + 2 = 6$

Answer: $f(3) = 6$

Q3. 3 identical green shirts + 3 identical red shirts distributed to 6 children (one each). How many ways?

What is given:

- 3 green shirts — all identical (same)
- 3 red shirts — all identical (same)
- 6 children, each gets exactly 1 shirt

Key Idea: *Shirts of the same colour are identical, so only decide WHICH 3 children get green. The rest automatically get red.*

Step-by-Step:

Step 1: Since shirts are identical, we only need to CHOOSE which 3 children get green shirts

Step 2: The remaining 3 children automatically get red shirts — no extra step needed

Step 3: Ways to choose 3 children out of 6 = 6C_3

Step 4: ${}^6C_3 = (6 \times 5 \times 4) / (3 \times 2 \times 1) = 120 / 6 = 20$

Why?

- We use Combination (not permutation) because shirts of same colour are identical
- Giving green shirt 1 vs green shirt 2 to the same child makes NO difference
- Example: children A,B,C,D,E,F — choosing A,B,C for green is ONE way; choosing A,B,D for green is ANOTHER way

Answer: 20 ways

Q4. $7^x - 7^{(x-2)} = 48 \times 7^6$. Find x.

What is given:

- Equation: $7^x - 7^{(x-2)} = 48 \times 7^6$

Key Idea: *Take the smaller power ($7^{(x-2)}$) as common factor from the left side.*

Step-by-Step:

Step 1: Take $7^{(x-2)}$ common from left side: $7^{(x-2)} \times (7^2 - 1)$

Step 2: Calculate inside bracket: $7^2 - 1 = 49 - 1 = 48$

Step 3: Left side becomes: $7^{(x-2)} \times 48$

Step 4: So: $7^{(x-2)} \times 48 = 48 \times 7^6$

Step 5: Cancel 48 from both sides: $7^{(x-2)} = 7^6$

Step 6: Since bases are equal, powers must be equal: $x - 2 = 6$

Step 7: $x = 6 + 2 = 8$

Answer: $x = 8$

Q5. Integers from 180 to 300 (inclusive) leaving remainder 2 when divided by both 15 and 9.

What is given:

- Range: 180 to 300 inclusive
- Condition: remainder 2 when divided by 15 AND by 9

Key Idea: *The number = (common multiple of 15 and 9) + 2. Find LCM first.*

Step-by-Step:

Step 1: LCM of 15 and 9: $15 = 3 \times 5$, $9 = 3 \times 3 \rightarrow \text{LCM} = 3 \times 3 \times 5 = 45$

Step 2: So required numbers are of the form: $45k + 2$

Step 3: Find multiples of 45 in range 180 to 300: $45 \times 4 = 180$, $45 \times 5 = 225$, $45 \times 6 = 270$

Step 4: Add 2 to each: $180 + 2 = 182$, $225 + 2 = 227$, $270 + 2 = 272$

Step 5: Check $272 + 45 = 317 > 300$, so stop here

Answer: 182, 227, 272 (3 numbers)

Q6. If the diameter of a circle increases by 50%, by what percent does the area increase?

What is given:

- Diameter increases by 50%
- Find: percentage increase in area

Key Idea: *Area depends on radius squared. If radius increases, area increases by the square of that ratio.*

Step-by-Step:

Step 1: Take original diameter = 10 $\rightarrow$ radius = 5

Step 2: Original area = $\pi \times r^2 = \pi \times 25$

Step 3: New diameter = $10 + 50\% \text{ of } 10 = 15 \rightarrow$ new radius = 7.5

Step 4: New area = $\pi \times 7.5^2 = \pi \times 56.25$

Step 5: Increase in area = $56.25 - 25 = 31.25$

Step 6: Percentage increase = $(31.25 / 25) \times 100 = 125\%$

Why?

- Shortcut: if radius multiplies by 1.5, area multiplies by $1.5^2 = 2.25$
- So area increases by $2.25 - 1 = 1.25 = 125\%$

Answer: 125%

Q7. $\sqrt{\sqrt{\sqrt{3x}}} = \sqrt[4]{2x}$, $x > 0$. Find x .

What is given:

- Left side: three nested square roots of $3x$
- Right side: 4th root of $2x$

Key Idea: *Convert roots to powers (fractions). Then raise both sides to a power to remove fractions.*

Step-by-Step:

Step 1: Left side: $\sqrt{\sqrt{\sqrt{3x}}} = (3x)^{(1/2 \times 1/2 \times 1/2)} = (3x)^{(1/8)}$

Step 2: Right side: 4th root of $2x = (2x)^{(1/4)} = (2x)^{(2/8)}$

Step 3: Raise both sides to the power 8 to clear fractions:

Step 4: $(3x)^1 = (2x)^2$

Step 5: $3x = 4x^2$

Step 6: Divide both sides by x ($x > 0$, so x is not 0): $3 = 4x$

Step 7: $x = 3/4$

Answer: $x = 3/4$

Q8. A group with average age 16 joins a class. The class average drops from 18 to 17. Find (joined : original).

What is given:

- Original class average = 18
- New students average = 16
- Combined average = 17

Key Idea: *Set up an equation using total age = average $\times$ number of students.*

Step-by-Step:

Step 1: Let original number of students = n , new students who joined = m

Step 2: Total age of original class = $18n$

Step 3: Total age of new students = $16m$

Step 4: Combined total age = $17(n + m)$

Step 5: So: $18n + 16m = 17(n + m)$

Step 6: Expand right side: $18n + 16m = 17n + 17m$

Step 7: $18n - 17n = 17m - 16m$

Step 8: $n = m$

Step 9: So joined : original = $m : n = 1 : 1$

Answer: 1 : 1

Q9. x = sum of all 3-digit numbers using distinct nonzero digits a, b, c exactly once. Largest integer that always divides x ?

What is given:

- Digits: a, b, c (all different, all nonzero)
- Form all possible 3-digit numbers using each digit exactly once

Key Idea: Count how many times each digit appears in each position (hundreds, tens, units).

Step-by-Step:

Step 1: Total 3-digit numbers possible = $3! = 6$ numbers

Step 2: These 6 numbers are: $abc, acb, bac, bca, cab, cba$

Step 3: Each digit (a, b, c) appears exactly 2 times in the hundreds place

Step 4: Each digit appears exactly 2 times in the tens place

Step 5: Each digit appears exactly 2 times in the units place

Step 6: Sum = $2(a+b+c) \times 100 + 2(a+b+c) \times 10 + 2(a+b+c) \times 1$

Step 7: Sum = $2(a+b+c) \times (100+10+1)$

Step 8: Sum = $2(a+b+c) \times 111 = 222(a+b+c)$

Step 9: Since $(a+b+c)$ changes with different digits, the fixed guaranteed divisor is 222

Answer: 222

Q10. Sophie Germain prime: prime p where $2p+1$ is also prime. Product of all possible units digits of Sophie Germain primes > 5 ?

What is given:

- Sophie Germain prime: p is prime AND $2p+1$ is also prime
- Find all possible UNITS DIGITS for primes greater than 5

Key Idea: Primes greater than 5 can only end in 1, 3, 7 or 9. Check which of these can be Sophie Germain primes.

Step-by-Step:

Step 1: Any prime > 5 must end in 1, 3, 7, or 9 (ending 0,2,4,5,6,8 means divisible by 2 or 5)

Step 2: Check ending 1: e.g. $p=11 \rightarrow 2 \times 11+1=23$ (prime) ✓ $\rightarrow$ units digit 1 is POSSIBLE

Step 3: Check ending 3: e.g. $p=23 \rightarrow 2 \times 23+1=47$ (prime) ✓ $\rightarrow$ units digit 3 is POSSIBLE

Step 4: Check ending 7: $2p+1$ ends in $2 \times 7+1=15 \rightarrow$ always ends in 5 $\rightarrow$ divisible by 5 $\rightarrow$ never prime ✗

Step 5: Check ending 9: e.g. $p=29 \rightarrow 2 \times 29+1=59$ (prime) ✓ $\rightarrow$ units digit 9 is POSSIBLE

Step 6: Possible units digits: 1, 3, 9

Step 7: Product = $1 \times 3 \times 9 = 27$

Answer: 27

Section 2 – Programming (Predict the Output)

Q11. Java – Ternary operator with odd number

```
public static String fun(int n) {  
    return (n % 2 == 0) ? "Success" : "Failure";  
}  
public static void main(String[] args) {  
    int n = 3;  
    String result = fun(n);  
    System.out.println(result);  
}
```

What is given:

- $n = 3$
- Function checks if n is even using $n \% 2 == 0$

Key Idea: % is the modulo (remainder) operator. If n is even, remainder is 0. If odd, remainder is 1.

Step-by-Step:

Step 1: $n = 3$

Step 2: $n \% 2 = 3 \% 2 = 1$ (not 0)

Step 3: Condition ($n \% 2 == 0$) is FALSE

Step 4: Ternary: false condition $\rightarrow$ returns 'Failure'

Step 5: System.out.println prints 'Failure'

Answer: Failure

Q12. C – Pre/post increment on array elements

```
int a[] = {5, 1, 15, 20, 25};  
int i, j, m;  
i = 1;  
i = ++a[i];  
j = a[i]++;  
m = a[i++];  
printf("%d %d %d", i, j, m);
```

What is given:

- Array $a = \{5, 1, 15, 20, 25\}$
- $i = 1$
- Three operations on array using i

Key Idea: $++a[i]$ means increment the array element FIRST. $a[i]++$ means use the value FIRST, then increment.

Step-by-Step:

Step 1: Start: $a = \{5, 1, 15, 20, 25\}$, $i = 1$

Step 2: Line 1: $i = ++a[i] \rightarrow$ first increment $a[1]$: $a[1]$ becomes $1+1=2$, then $i = 2$. Now $a = \{5, 2, 15, 20, 25\}$

Step 3: Line 2: $j = a[i]++ \rightarrow i$ is 2, so $j = a[2] = 15$, then $a[2]$ becomes 16. $j = 15$. Now $a = \{5, 2, 16, 20, 25\}$

Step 4: Line 3: $m = a[i++] \rightarrow i$ is 2, so $m = a[2] = 16$, then i becomes 3. $m = 16$

Step 5: Final values: $i = 3$, $j = 15$, $m = 16$

Answer: 3 16 16

Q13. C – Redeclaring static variable three times in same scope

```
void fun() {
    static int i;
    static int i = 25;    // ERROR - redeclaration
    static int i;
    printf("%d\n", i);
}
int main() {
    static int i;
    printf("%d\n", i);
    fun();
}
```

What is given:

- Inside fun(), 'static int i' is written 3 times
- One has an initializer (= 25), two do not

Key Idea: In C, you cannot declare the same variable more than once in the same scope — even if it is static.

Step-by-Step:

Step 1: First declaration: static int i; — OK so far

Step 2: Second declaration: static int i = 25; — ERROR: 'i' already declared in this scope

Step 3: C does not allow redefinition of a variable in the same block

Step 4: The compiler will reject the program

Answer: Compilation Error

Q14. C – Empty statement after if (semicolon trap)

```
int main() {
    int i = 3;
    if (i == 2); i = 0;    // semicolon = empty if body!
    if (i == 3) i++;
    else i += 2;
}
```

What is given:

- i = 3 at start
- Two if statements, one with a semicolon after it

Key Idea: if (condition); means the if body is EMPTY (the semicolon). The next line always runs regardless of condition.

Step-by-Step:

Step 1: i = 3

Step 2: if (i == 2); → this semicolon makes the if body empty. Nothing happens for the if.

Step 3: i = 0; → this line is NOT inside the if. It ALWAYS runs. Now i = 0

Step 4: if (i == 3) → i is 0, not 3. Condition is FALSE

Step 5: So else runs: i = i + 2 = 0 + 2 = 2

Step 6: No printf in the program, so nothing is printed

Answer: No output (i ends as 2)

Q15. C – Partial string reversal (only first len=7 characters)

```
void fun(char str[], int len) {
    char temp;
    for (int a = 0; a < len / 2; a++) {
        temp = str[a];
        str[a] = str[len - a - 1];
        str[len - a - 1] = temp;
    }
}

int main() {
    char str[] = "zohocorporation";
    int len = 7;           // only reverses first 7 chars!
    fun(str, len);
    printf("%s\n", str);
}
```

What is given:

- String = 'zohocorporation'
- fun() reverses only the first 'len' characters
- len is passed as 7 (not full length)

Key Idea: *The function reverses only the first 7 characters, not the entire string.*

Step-by-Step:

Step 1: Full string: z o h o c o r p o r a t i o n

Step 2: First 7 characters: z o h o c o r

Step 3: Reversed: r o c o h o z

Step 4: Remaining characters (position 7 onwards): p o r a t i o n — UNCHANGED

Step 5: Final string = 'rocohoz' + 'poration' = 'rocohozporation'

Answer: rocohozporation

Q16. Mixed Java + C code — System.out.print and printf together

```
int a, b;
for (int a=6, b=4; a<=24; a++) {
    if (a % b == 0) break;
}
System.out.print(a);    // Java syntax
printf("%d\n", i);      // C syntax - 'i' not declared
```

What is given:

- Code uses Java syntax (System.out.print)
- Code also uses C syntax (printf)
- Variable 'i' is used but never declared outside the loop

Key Idea: *You cannot mix Java and C in the same program. They are completely different languages.*

Step-by-Step:

Step 1: System.out.print is Java syntax

Step 2: printf is C syntax

Step 3: These two languages cannot be compiled together

Step 4: Also: variable 'a' declared inside for loop dies when loop ends; outer 'a' is uninitialized

Step 5: Variable 'i' is used in printf but never declared

Step 6: Program cannot compile

Answer: Compilation Error

Q17. C – Recursive function with threshold condition

```
static int q(int x, int y) {
    int i = x * y;
    return i < 50 ? i : q(2, 3);
}
int main() {
    int a = 20, b = 4;
    int result = q(a, b);
    printf("Result: %d\n", result);
}
```

What is given:

- $q(x, y)$ returns $x*y$ if result < 50 , else calls $q(2,3)$
- Called with $a=20, b=4$

Key Idea: Trace the function calls one by one.

Step-by-Step:

- Step 1: Call $q(20, 4)$: $i = 20 \times 4 = 80$
- Step 2: Is $80 < 50$? NO $\rightarrow$ so call $q(2, 3)$ instead
- Step 3: Call $q(2, 3)$: $i = 2 \times 3 = 6$
- Step 4: Is $6 < 50$? YES $\rightarrow$ return 6
- Step 5: So result = 6

Answer: Result: 6

Q18. C – do-while loop with double increment (++i appears twice per iteration)

```
int fun(int val) {
    int ans = val;
    for (int i = 0; i < val; i++)
        ans = ans + val * val;
    return ans;
}
int main() {
    int a[] = {0, 1, 2, 3, 4};
    int i = 0;
    do {
        printf("%d\n", fun(a[++i]));
    } while (++i < 5);
}
```

What is given:

- Array $a = \{0, 1, 2, 3, 4\}$
- $fun(val)$ calculates $val + val*val*val$ ($val + val^3$)
- i is incremented TWICE per loop iteration

Key Idea: i is incremented once inside the loop ($a[++i]$) and once in the while condition ($++i$). So i jumps by 2 each time.

Step-by-Step:

- Step 1: $fun(val)$ formula: starts with $ans=val$, adds $val*val$ each iteration val times $\rightarrow ans = val + val*val*val = val + val^3$
- Step 2: i starts at 0
- Step 3: Iteration 1: $a[++i] = a[1] = 1 \rightarrow fun(1) = 1 + 1 = 2 \rightarrow$ printed. Then $++i \rightarrow i = 2$. $2 < 5$ so continue.
- Step 4: Iteration 2: $a[++i] = a[3] = 3 \rightarrow fun(3) = 3 + 27 = 30 \rightarrow$ printed. Then $++i \rightarrow i = 4$. $4 < 5$ so continue.
- Step 5: Iteration 3: $a[++i] = a[5] \rightarrow$ index 5 does not exist (array has 0-4) $\rightarrow$ undefined behaviour / garbage

Answer: 2, 30, then garbage (undefined behaviour)

Q19. C – Index chasing through two arrays (7 characters)

```
char c[] = {'z','i','o','s','a','n','b'};
int ind[] = {4, 3, 6, 1, 0, 2, 5};
int i = 5;
i = ind[i];
char b = c[i];
i = i - ind[i];
i = ind[4] + 1;
char a = c[i];
printf("%c %c\n", a, b);
```

What is given:

- char c[] = {z, i, o, s, a, n, b} (index 0 to 6)
- int ind[] = {4, 3, 6, 1, 0, 2, 5} (index 0 to 6)
- Trace each line carefully

Key Idea: *Each line uses the current value of i as an index into ind[] or c[]. Trace one line at a time.*

Step-by-Step:

Step 1: i = 5

Step 2: i = ind[5] = 2 → i is now 2

Step 3: b = c[2] = 'o' → b = 'o'

Step 4: i = i - ind[i] = 2 - ind[2] = 2 - 6 = -4 (but this value is immediately replaced below, so ignore)

Step 5: i = ind[4] + 1 = 0 + 1 = 1 → i is now 1

Step 6: a = c[1] = 'i' → a = 'i'

Step 7: Print a then b: 'i' then 'o'

Answer: i o

Q20. C – Index chasing through two arrays (6 characters)

```
char a[] = {'a', 'o', 'o', 'k', 'u', 'b'};
int ind[] = {1, 2, 3, 3, 2, 4, 5};
int n = 2;
char l = a[ind[n]];
n = ind[n];
char i = a[n - 1];
n = n + ind[n];
char j = a[n - 1];
n = ind[n] - 1;
char k = a[n];
printf("%c %c %c %c\n", i, j, k, l);
```

What is given:

- char a[] = {a, o, o, k, u, b} (index 0 to 5)
- int ind[] = {1, 2, 3, 3, 2, 4, 5} (index 0 to 6)
- n starts at 2

Key Idea: Trace each line one by one, updating n each time.

Step-by-Step:

Step 1: n = 2

Step 2: l = a[ind[2]] = a[3] = 'k' → l = 'k'

Step 3: n = ind[2] = 3 → n is now 3

Step 4: i = a[n-1] = a[2] = 'o' → i = 'o'

Step 5: n = n + ind[n] = 3 + ind[3] = 3 + 3 = 6 → n is now 6

Step 6: j = a[n-1] = a[5] = 'b' → j = 'b'

Step 7: n = ind[n] - 1 = ind[6] - 1 = 5 - 1 = 4 → n is now 4

Step 8: k = a[n] = a[4] = 'u' → k = 'u'

Step 9: Print i, j, k, l: 'o', 'b', 'u', 'k'

Answer: o b u k